

Technical Answers & Project Implementation Roadmap

This document provides detailed answers to your 7 queries regarding the ****AI-Based Precision Fertigation & Crop Health Monitoring System****. It outlines exact technical workflows, architectural code examples, and step-by-step instructions for transitioning from the prototype to a production-ready system.

1. Removing Synthetic Data & Transitioning to Real-World Datasets

Current State

Currently, the system uses baseline placeholder/synthetic data generators in:

- `hardware/simulator.py` (simulates sensor telemetry streams).
- `backend/main.py` (provides fallback default dict values when the database is empty).

Step-by-Step Transition Plan

A. Data Ingestion & Database Migration Script

Create an automated ingestion script (`backend/database/ingest\_real\_data.py`) to parse real-world agricultural datasets (e.g., PlantVillage, Kaggle Crop Recommendation/Yield datasets, or custom field sensor CSV/JSON logs) into SQLite/PostgreSQL.

```
```python
backend/database/ingest_real_data.py
import pandas as pd
from sqlalchemy.orm import Session
from database.connection import engine, SessionLocal
from database.models import SensorReading, ImageParameter, PredictionLog

def ingest_csv_dataset(csv_filepath: str):
 db: Session = SessionLocal()
 df = pd.read_csv(csv_filepath)

 for _, row in df.iterrows():
 sensor = SensorReading(
 soil_moisture=float(row['soil_moisture']),
 ambient_temp=float(row['temperature']),
 humidity=float(row['humidity']),
 light_intensity=float(row['light_intensity'])
)
 db.add(sensor)

 db.commit()
 db.close()
 print(f"Successfully ingested {len(df)} real-world records.")
...
```
```

B. Updating Backend APIs

Remove synthetic dictionary fallbacks in `backend/main.py` so that endpoints directly query active database tables or return empty indicator states when data is missing:

```
```python
```

```

@app.get("/api/dashboard/updates")
def get_live_updates(db: Session = Depends(get_db)):
 latest_sensor =
db.query(SensorReading).order_by(SensorReading.timestamp.desc()).first()
 latest_image =
db.query(ImageParameter).order_by(ImageParameter.timestamp.desc()).first(
)

 if not latest_sensor:
 raise HTTPException(status_code=404, detail="No real telemetry
data available in database.")

 return {
 "soil_moisture": latest_sensor.soil_moisture,
 "ambient_temp": latest_sensor.ambient_temp,
 "humidity": latest_sensor.humidity,
 "light_intensity": latest_sensor.light_intensity,
 "canopy_area": latest_image.canopy_area if latest_image else 0.0,
 "excess_green_index": latest_image.excess_green_index if
latest_image else 0.0,
 "timestamp": latest_sensor.timestamp
 }
...

```

#### #### C. Frontend UI Behavior

Update `frontend/app.js` to handle empty states gracefully (e.g., displaying `"Awaiting Field Data..."`) instead of hardcoding initial fallback metrics.

---

## ## 2. Integrating Teammate's Trained ML/DL Model

### ### Standard Export Formats

Ask your teammate to export the trained model in one of the following production formats:

- `**ONNX (`.onnx`)**` `*(Recommended)*`: Platform-agnostic, lightweight, fast inference across both PC and Raspberry Pi.
- `**PyTorch (`.pt` / `.pth`)**`: Recommended if using PyTorch natively on backend server.
- `**TensorFlow / Keras (`.h5` / `.keras`)**` or `**SavedModel format**`.
- `**Scikit-Learn / Joblib (`.joblib` / `.pkl`)**`: For tabular ML models (e.g., Random Forest, XGBoost).

### ### Integration Architecture in Backend

...

```

backend/
├── models/
│ ├── fertigation_model.onnx <-- Teammate's trained model file
│ └── tabular_scaler.joblib <-- Preprocessing scaler
├── ml_engine/
│ ├── __init__.py
│ └── inference.py <-- Model loader & prediction
function
...

```

### ### Implementation Code (`backend/ml\_engine/inference.py`)

```

```python
import onnxruntime as ort
import numpy as np
import cv2
import joblib

MODEL_PATH = "backend/models/fertigation_model.onnx"
SCALER_PATH = "backend/models/tabular_scaler.joblib"

# Load ONNX session once on startup
session = ort.InferenceSession(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)

def preprocess_image(image_bytes: bytes) -> np.ndarray:
    # Decode and resize to model's input shape (e.g., 224x224)
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    img = cv2.resize(img, (224, 224))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img.astype(np.float32) / 255.0 # Normalize [0, 1]
    img = np.transpose(img, (2, 0, 1)) # HWC to CHW format
    return np.expand_dims(img, axis=0) # Batch dimension (1, 3, 224,
224)

def run_multi_task_inference(image_bytes: bytes, sensor_vector: list):
    # 1. Preprocess inputs
    img_tensor = preprocess_image(image_bytes)
    scaled_sensors = scaler.transform([sensor_vector])

    # 2. Run ONNX Inference
    inputs = {
        session.get_inputs()[0].name: img_tensor,
        session.get_inputs()[1].name: scaled_sensors.astype(np.float32)
    }
    outputs = session.run(None, inputs)

    # Extract outputs (e.g., outputs[0] = water_ml, outputs[1] = fert_g,
outputs[2] = disease_prob)
    return {
        "water_ml": float(outputs[0][0]),
        "fertilizer_g": float(outputs[1][0]),
        "disease_prob": float(outputs[2][0])
    }
...

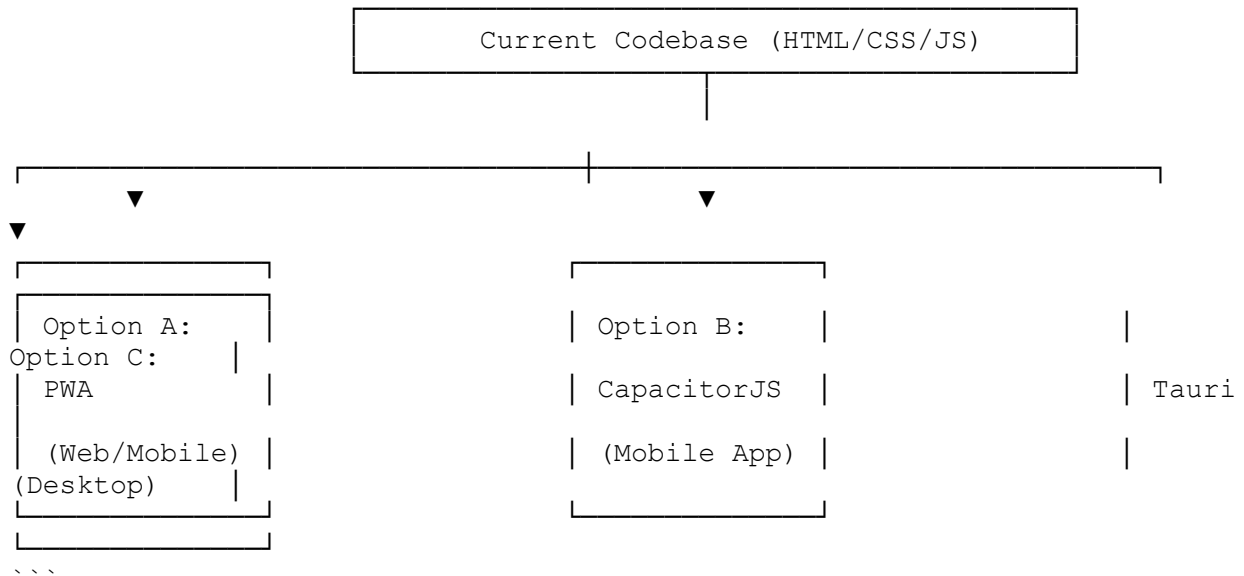
---

## 3. Upgrading to Mobile & Desktop Apps

### Feasibility: **Yes, very straightforward!**
Because the existing frontend is built using standard web technologies
(HTML5, CSS3, JavaScript) communicating via a REST API, converting it
into mobile and desktop applications requires minimal codebase changes.

### Recommended Packaging Options
...

```



Deployment Target	Recommended Tool	Effort	Key Features
Mobile (Android/iOS)	Progressive Web App (PWA)	Very Low	Installable directly from browser, offline caching, push notifications, zero app store approval needed.
Mobile Native App (.apk / .aab)	CapacitorJS	Low	Wraps web code into native Android Studio / Xcode projects. Accesses hardware camera, GPS, Bluetooth.
Desktop (.exe / .AppImage)	Tauri / Electron	Low	Converts frontend + Python backend into a single executable installer for Windows, Mac, Linux, and Pi Desktop.

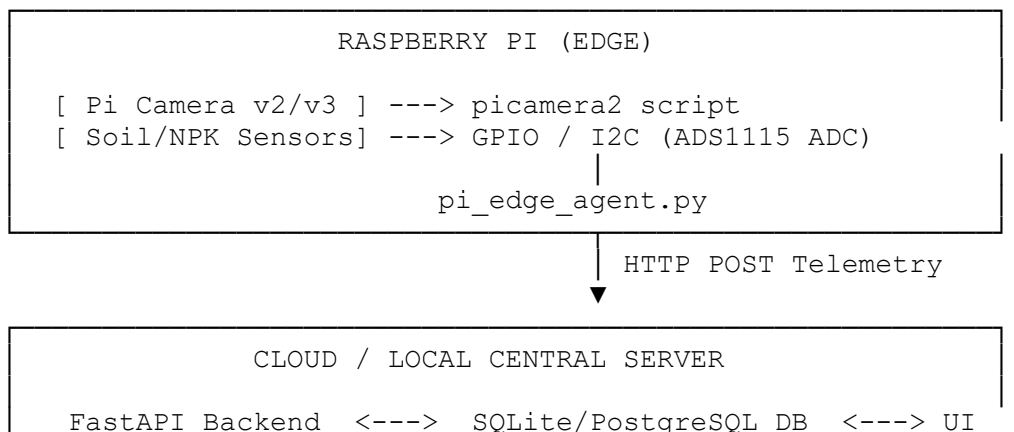
4. Raspberry Pi OS & Pi Camera Integration

Should you work on Raspberry Pi OS now?

- **Workflow Recommendation**: Continue coding and building features on your PC. Deploy to the Raspberry Pi for hardware field testing via SSH/Git.

Raspberry Pi Deployment Architecture

...



```

|_____
...

### Raspberry Pi Edge Script (`hardware/pi_edge_agent.py`)

```python
import time
import requests
from picamera2 import Picamera2

Initialize Pi Camera v2/v3 using modern picamera2 library
picam2 = Picamera2()
picam2.start()

SERVER_URL = "http://<YOUR_SERVER_IP>:8000/api/ingest/edge-data"

def capture_and_read_sensors():
 # 1. Capture canopy image
 image_path = "/tmp/canopy_latest.jpg"
 picam2.capture_file(image_path)

 # 2. Read physical sensors via I2C/GPIO (Example values)
 sensor_payload = {
 "soil_moisture": 46.2,
 "ambient_temp": 28.1,
 "humidity": 65.0,
 "light_intensity": 15400
 }

 # 3. Transmit image + sensors to central backend API
 with open(image_path, "rb") as img_file:
 files = {"image": img_file}
 response = requests.post(SERVER_URL, data=sensor_payload,
files=files)
 print("Telemetry transmission status:", response.status_code)

if __name__ == "__main__":
 while True:
 capture_and_read_sensors()
 time.sleep(300) # Capture every 5 minutes
...

5. Specialized Agricultural AI Assistant (Chatbot)

Architecture
Add a floating AI Chatbot assistant in the application UI that answers
user queries (e.g., "Why is my nitrogen recommendation high today?",
"How to treat Early Blight in Tomato?").

...

User Query ---> UI Chat Drawer ---> FastAPI /api/chat ---> Context
Generator ---> LLM Engine (Gemini / Local Ollama)

|
Injects Live Sensor

State
...

```

```

Backend Implementation (`backend/chatbot.py`)

```python
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from database.connection import get_db
from database.models import SensorReading, PredictionLog
import google.generativeai as genai

router = APIRouter()
genai.configure(api_key="YOUR_GEMINI_API_KEY")

@router.post("/api/chat")
def agricultural_ai_assistant(user_message: str, db: Session =
Depends(get_db)):
    # 1. Fetch real-time farm context from DB
    latest_sensor =
db.query(SensorReading).order_by(SensorReading.timestamp.desc()).first()
    latest_pred =
db.query(PredictionLog).order_by(PredictionLog.timestamp.desc()).first()

    system_context = f"""
    You are 'AgriSense AI', an expert agricultural assistant specialized
in precision fertigation.
    Current Live Farm Metrics:
    - Soil Moisture: {latest_sensor.soil_moisture if latest_sensor else
'N/A'}%
    - Ambient Temperature: {latest_sensor.ambient_temp if latest_sensor
else 'N/A'}°C
    - Health Score: {latest_pred.health_score if latest_pred else
'N/A'}/100
    - Recommended Water: {latest_pred.water_requirement_ml if latest_pred
else 'N/A'} ml
    - Recommended Fertilizer: {latest_pred.fertilizer_npk_dose_g if
latest_pred else 'N/A'} g

    Answer the user's question concisely using this real-time context.
    """

    model = genai.GenerativeModel("gemini-1.5-flash")
    response = model.generate_content(f"{system_context}\nUser Question:
{user_message}")

    return {"reply": response.text}
...
---

```

6. Bilingual Interface Support (English & Tamil - தமிழ்)

```

### Implementation Design
Using JSON translation dictionaries and a dynamic frontend switcher in
`frontend/index.html` and `frontend/app.js`.

### Translation Dictionaries

#### English Dictionary (`frontend/i18n/en.json`)

```

```

```json
{
 "app_title": "AI Precision Fertigation",
 "soil_moisture": "Soil Moisture",
 "ambient_temp": "Ambient Temperature",
 "health_score": "Crop Health Score",
 "water_req": "Water Requirement",
 "fert_req": "Fertilizer Dosage",
 "execute_action": "Execute Action"
}
```

#### Tamil Dictionary (`frontend/il8n/ta.json`)
```json
{
 "app_title": "செயற்கை நுண்ணறிவு பாசன அமைப்பின் முனையம்",
 "soil_moisture": "மண் ஈரப்பதம்",
 "ambient_temp": "சுற்றுப்புற வெப்பநிலை",
 "health_score": "பயிர் ஆரோக்கிய நிலை",
 "water_req": "தேவையான நீரின் அளவு",
 "fert_req": "பரிந்துரைக்கப்பட்ட உர அளவு",
 "execute_action": "செயல்படுத்து"
}
```

```

```

### Frontend Switcher Logic (`frontend/app.js`)

```

```

```javascript
let currentLang = localStorage.getItem('user_language') || 'en';

async function setLanguage(lang) {
 currentLang = lang;
 localStorage.setItem('user_language', lang);
 const response = await fetch(`./il8n/${lang}.json`);
 const translations = await response.json();

 document.querySelectorAll('[data-il8n]').forEach(element => {
 const key = element.getAttribute('data-il8n');
 if (translations[key]) {
 element.textContent = translations[key];
 }
 });
}
```

```

```

---

```

```

## 7. Current ML Algorithms & CNN / RNN Clarification

```

```

### Current Status in Existing Codebase

```

```

- In the baseline prototype built so far, the application uses rule-based simulation functions in `hardware/simulator.py` and `backend/main.py` to establish the API structure, database tables, and UI dashboard.
- Actual CNN and RNN models are not currently running in the baseline script yet.

```

— — —



- ****Input****: Canopy images captured by the Raspberry Pi Camera.

1. Feature extraction of Excess Green Index ($\$ExG = 2G - R - B\$$) and leaf segmentation.
2. Canopy coverage area estimation.
3. Disease classification (e.g., detecting Early Blight or Nutrient Deficiency spots using ResNet-50 or EfficientNet).

```
- **Input**: Continuous time-series sensor streams (Soil moisture, humidity, temperature, sunlight over hours/days).
```

1. Modeling non-linear soil drying curves and evapotranspiration (\$ET_0\$).
2. Predicting daily water requirement (\$m_l\$) and optimal N-P-K fertilizer dosage (\$q\$).

— — —

Summary Next Steps Checklist

1. ****Upload Teammate's Model****: Save model file as ``.onnx`` or ``.pt`` into ``backend/models/``.
2. ****Import Real Dataset****: Run CSV data ingestion script to replace synthetic baseline data.
3. ****Enable Bilingual Switcher****: Add ``i18n`` JSON files and dropdown selector to UI header.
4. ****Deploy Pi Edge Script****: Setup ``picamera2`` edge capture agent on Raspberry Pi.
5. ****Connect AI Chatbot****: Integrate Gemini API endpoint in ``/api/chat``.